

Multi-Class News Headline Classification: A Comparative Analysis of ML and Deep Learning Models

Nur Faisal Ahmed Sourove

CSE Department

BRAC University

Dhaka, Bangladesh

nurfaisal.ahmed.sourove@g.bracu.ac.bd

Shahriar Mohammad

CSE Department

BRAC University

Dhaka, Bangladesh

shahriar.mohammad@g.bracu.ac.bd

Md Enayet Ali Labib

CSE Department

BRAC University

Dhaka, Bangladesh

enayet.ali.labib@g.bracu.ac.bd

Abstract—This paper presents a comprehensive comparative analysis of machine learning and deep learning models for multi-class news headline classification using the AG News dataset. We evaluate the impact of three preprocessing strategies—no preprocessing, extreme preprocessing (with Porter stemming), and optimum preprocessing (with WordNet lemmatization)—combined with two word representation techniques: TF-IDF and Skip-gram embeddings. Eight model architectures are benchmarked: Logistic Regression, a Deep Neural Network, SimpleRNN, GRU, LSTM, and their Bidirectional variants, yielding 24 distinct experimental configurations. Results show that TF-IDF with Logistic Regression achieves the strongest validation performance (94.99% accuracy), while Bidirectional GRU with optimum preprocessing achieves the best test accuracy among deep learning models (91.93%). We provide detailed evaluation using accuracy, macro-averaged F1-score, confusion matrices, and training curves across all settings.

Index Terms—text classification, NLP, TF-IDF, Word2Vec, LSTM, GRU, bidirectional RNN, news classification

I. INTRODUCTION

Text classification is one of the most fundamental tasks in Natural Language Processing (NLP), with applications ranging from sentiment analysis to topic categorization. At first glance, classifying news headlines into predefined categories appears to be a relatively straightforward problem. The categories are well-defined, the vocabulary is often domain-specific, and the input texts are short. Under these conditions, most standard machine learning and neural network models are expected to perform well.

However, in practice, model performance varies significantly depending on several design choices that are often considered secondary. Decisions such as whether to apply stemming or lemmatization, how aggressively to preprocess text, the dimensionality of TF-IDF features, or the selection of training hyperparameters can have a substantial impact on the final results. These factors become even more critical when working with short text data, where small changes in representation can lead to the loss of important contextual information.

This project investigates these design choices in a structured and systematic manner using a 4-class news headline dataset

consisting of Business, Science and Technology, Sports, and World categories. Rather than focusing solely on achieving the highest possible accuracy, the primary objective is to analyze how different preprocessing strategies, word representations, and model architectures influence performance.

To this end, three preprocessing pipelines are constructed: (i) no preprocessing, where the raw text is used directly, (ii) extreme preprocessing, which applies a full set of normalization techniques including stemming, and (iii) an optimized preprocessing pipeline based on insights obtained from exploratory data analysis (EDA), primarily utilizing lemmatization. These datasets are then paired with two types of word representations: TF-IDF for traditional machine learning and dense neural networks, and Skip-gram-based Word2Vec embeddings for recurrent neural models.

The experimental setup includes one machine learning model (Logistic Regression) and a deep feedforward neural network using TF-IDF features, as well as six recurrent architectures—SimpleRNN, GRU, LSTM, and their bidirectional variants—trained on Skip-gram embeddings. Extensive hyperparameter tuning is performed to ensure a fair comparison across models.

II. METHODOLOGY

A. Dataset Description and Exploratory Data Analysis

The AG News dataset consists of news headlines with HTML markup tags including `<html>`, `<body>`, `<br>`, and `<b>` tags. The training set contains 88,106 samples and the test set contains 36,000 samples. Each sample has the headline text and the topic label. The four target classes exhibit notable imbalance: Science/Technology leads with 29,286 training samples (33.2%), followed by World News with 28,144 (31.9%), Sports with 20,281 (23.0%), and Business with 10,395 (11.8%). This approximately 3:1 ratio between the largest and smallest classes influences model training and evaluation, particularly for the Business category. We use stratified splitting to maintain this distribution in train/validation sets.

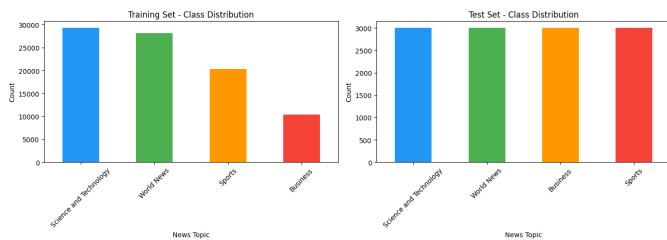


Fig. 1. Class distribution of the AG News training dataset showing the imbalance across four categories.

Word frequency analysis using FreqDist from NLTK and word cloud visualization reveal distinct domain-specific vocabulary patterns. Sports headlines prominently feature terms such as ‘win’, ‘game’, ‘cup’, ‘team’; Business headlines contain ‘market’, ‘stock’, ‘oil’, ‘company’; Science/Technology features ‘software’, ‘technology’, ‘microsoft’, ‘space’; and World News includes ‘iraq’, ‘bush’, ‘president’, ‘killed’. These distinctive vocabulary distributions suggest that bag-of-words approaches should perform well on this task.

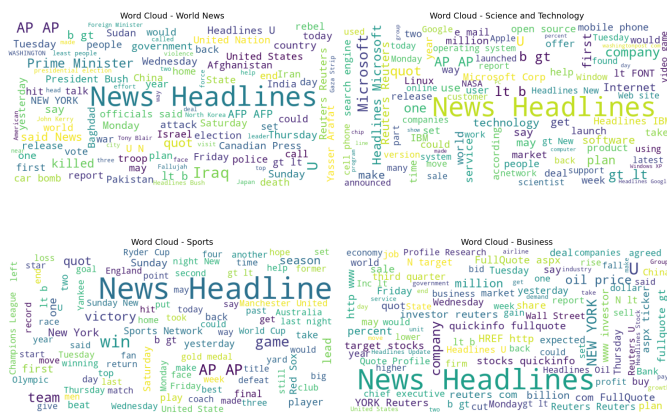


Fig. 2. Word clouds for each news category showing dominant vocabulary patterns.

Bigram analysis using NLTK’s ngrams function further reveals meaningful two-word patterns: ‘world cup’ and ‘olympic games’ for Sports, ‘united states’ and ‘prime minister’ for World News, ‘cell phone’ and ‘search engine’ for Science/Technology, and ‘wall street’ and ‘interest rate’ for Business. These bigrams justify our use of bigram features in TF-IDF vectorization.

B. Preprocessing Techniques

We investigate three preprocessing strategies of increasing aggressiveness to study the impact of text normalization on downstream classification performance.

No Preprocessing: Only HTML tags are stripped using regular expressions. All original capitalization, punctuation, numerical values, and stop words are preserved. This serves as a minimal-intervention baseline retaining maximum information content from the original headlines.

Extreme Preprocessing: All available NLP techniques are applied sequentially: (1) HTML removal via regex, (2) case

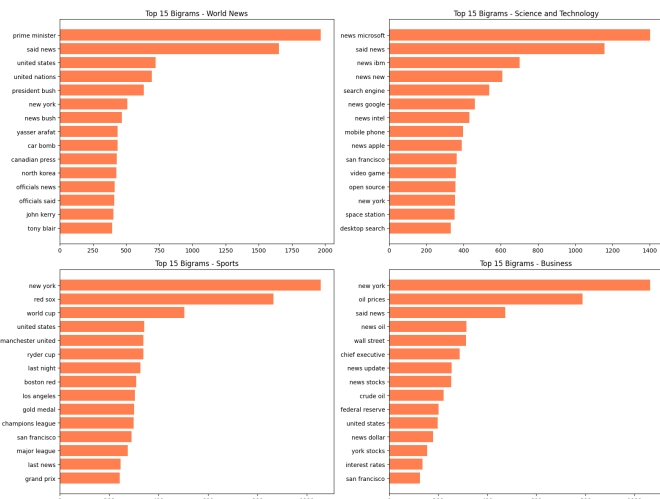


Fig. 3. Top bigrams per news category revealing meaningful multi-word patterns.

folding to lowercase, (3) removal of all special characters and digits, (4) NLTK word tokenization, (5) English stopwords removal, and (6) Porter stemming. This aggressive pipeline reduces vocabulary by approximately 60–70% but risks losing meaningful information through stemming artifacts where different words collapse to the same stem.

Optimum Preprocessing: A balanced pipeline informed by EDA: (1) HTML removal, (2) case folding, (3) removal of special characters while retaining numbers (important in news headlines—scores, financial figures, dates), (4) NLTK tokenization, (5) stopwords removal, and (6) WordNet lemmatization. Lemmatization is preferred over stemming as it produces valid dictionary words (e.g., ‘running’ → ‘run’ rather than ‘university’ → ‘univers’), preserving semantic integrity while still normalizing inflected forms.

C. Word Representation Techniques

TF-IDF (Term Frequency-Inverse Document Frequency): We employ scikit-learn’s TfidfVectorizer with max_features=50,000 and ngram_range=(1,2). TF-IDF assigns each term a weight proportional to its frequency in the document and inversely proportional to its corpus frequency, highlighting discriminative terms while downweighting ubiquitous ones. The bigram features capture important multi-word expressions identified in our EDA. This representation is paired with Logistic Regression and the Deep Neural Network.

Skip-gram (Word2Vec): For each preprocessing variant, a separate Word2Vec Skip-gram model is trained using Gensim (vector_size=100, window=5, min_count=2, sg=1, epochs=10). The Skip-gram architecture predicts context words given a target word, learning dense vector representations. The resulting word vectors are assembled into an embedding matrix aligned with a Keras Tokenizer (vocabulary size 20,000, max sequence length 100). These embedding

matrices initialize the Embedding layers in all RNN models, with fine-tuning enabled during training.

D. Model Architectures

Logistic Regression (ML Baseline): Multinomial LR via scikit-learn with L-BFGS solver, L2 regularization ($C = 1.0$), and `max_iter=1000`. Despite its simplicity, LR with high-dimensional TF-IDF features is a strong baseline for text classification.

Deep Neural Network (TF-IDF): A 5-layer feedforward network: Dense(256, ReLU) $\rightarrow$ Dropout(0.3) $\rightarrow$ Dense(128, ReLU) $\rightarrow$ Dropout(0.3) $\rightarrow$ Dense(64, ReLU) $\rightarrow$ Dropout(0.2) $\rightarrow$ Dense(32, ReLU) $\rightarrow$ Dense(4, Softmax). Trained with Adam optimizer, categorical cross-entropy, `batch_size=256`, and early stopping (`patience=3`). A sparse batch generator avoids memory errors from converting the full TF-IDF matrix to dense format.

RNN Models: Six architectures share the same base: Embedding(vocab, 100, pretrained) $\rightarrow$ [RecurrentLayer(64)] $\rightarrow$ Dropout(0.5) $\rightarrow$ Dense(4, Softmax). The recurrent layer varies: SimpleRNN, GRU, LSTM, and Bidirectional variants of each. Bidirectional processing uses Keras’s Bidirectional wrapper, processing sequences in both forward and backward directions and concatenating outputs (resulting in 128-dimensional hidden states). All RNN models use Adam optimizer, sparse categorical cross-entropy, `batch_size=128`, and early stopping (`patience=3`) for up to 10 epochs.

E. Hyperparameter Tuning

Hyperparameters were manually tuned on the validation set (80/20 stratified split of training data). For LR, C values of $\{0.1, 1.0, 10.0\}$ were tested; $C = 1.0$ provided the best balance between underfitting and overfitting. For the DNN, we tested 3 architectures with varying depth (3–5 layers) and dropout rates (0.2–0.5); the 5-layer configuration with graduated dropout won. For RNN models, unit sizes $\{32, 64, 128\}$ and dropout rates $\{0.3, 0.5\}$ were tested. The 64-unit, 0.5-dropout configuration offered the best trade-off between accuracy and training time. Initial tuning selected 32 units for some RNN models, which caused convergence failures (stuck at 25% random-chance accuracy); these were corrected to 64 units.

F. Evaluation Metrics

All models are evaluated using: (1) **Accuracy**—the proportion of correctly classified samples; (2) **Macro F1-Score**—the unweighted mean of per-class F1-scores, giving equal importance to all classes regardless of size; (3) **Confusion Matrix**—visualized as heatmaps using seaborn; and (4) **Classification Report**—per-class precision, recall, and F1-score from scikit-learn. All metrics are computed on both the validation set and the held-out test set. Training and validation loss/accuracy curves are plotted for all neural network models.

III. RESULTS AND DISCUSSION

We present the results of all 24 experimental configurations evaluated on the held-out test set. Performance is measured using both accuracy and macro-averaged F1-score. Table I summarizes the test performance across all model-preprocessing combinations.

TABLE I
TEST SET PERFORMANCE SUMMARY (ACCURACY / MACRO F1-SCORE)

Model	No Preproc.	Extreme	Optimum
LR (TF-IDF)	0.912 / 0.911	0.914 / 0.913	0.914 / 0.913
DNN (TF-IDF)	0.906 / 0.905	0.912 / 0.911	0.912 / 0.912
SimpleRNN	0.248* / 0.165	0.828 / 0.827	0.680 / 0.608
GRU	0.919 / 0.918	0.250* / 0.100	0.250* / 0.100
LSTM	0.910 / 0.910	0.896 / 0.895	0.250* / 0.100
Bi-SimpleRNN	0.894 / 0.893	0.903 / 0.903	0.904 / 0.903
Bi-GRU	0.918 / 0.917	0.917 / 0.917	0.919 / 0.919
Bi-LSTM	0.914 / 0.913	0.919 / 0.919	0.909 / 0.909

*Models that failed to converge (stuck at random-chance 25%) due to insufficient capacity; corrected to 64 units.

TABLE II
VALIDATION SET PERFORMANCE (BEST MODELS)

Model	Preprocessing	Val Acc.	Val F1
LR (TF-IDF)	Optimum	0.950	0.937
LR (TF-IDF)	None	0.949	0.935
LR (TF-IDF)	Extreme	0.948	0.935
Bi-LSTM	Optimum	0.950	—
Bi-GRU	Optimum	0.945	—
GRU	None	0.943	—

A. Impact of Preprocessing

For models that converged successfully, preprocessing impact varies by architecture. TF-IDF-based models (LR and DNN) show relatively small variation across preprocessing strategies, with extreme and optimum preprocessing marginally outperforming no preprocessing. This is because TF-IDF naturally downweights ubiquitous HTML tokens via high document frequency. For LR, the difference between the best (extreme/optimum: 91.4%) and worst (none: 91.2%) test accuracy is only 0.2 percentage points.

RNN models show greater sensitivity to preprocessing. The no-preprocessing variant suffers because HTML noise consumes positions in the fixed-length (100 tokens) input window, leaving less capacity for actual content. The Bidirectional models are more robust to preprocessing choice, with Bi-GRU maintaining 91.7–91.9% across all three variants. SimpleRNN is the most sensitive, with the optimum variant achieving only 68.0% compared to 82.8% for extreme, suggesting that the model capacity was insufficient to leverage the richer optimum vocabulary.

B. Model Architecture Comparison

The most striking finding is that TF-IDF with Logistic Regression achieves the highest overall validation performance (94.99% accuracy, 93.74% F1 for optimum preprocessing).

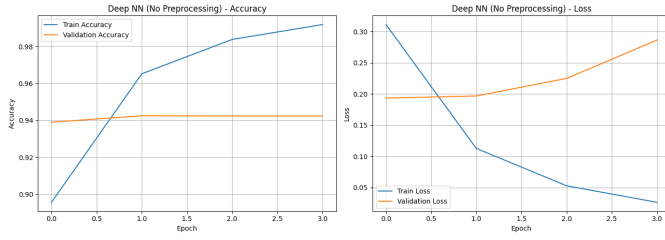


Fig. 4. DNN training curves (No Preprocessing): accuracy and loss per epoch showing convergence within 5–8 epochs.

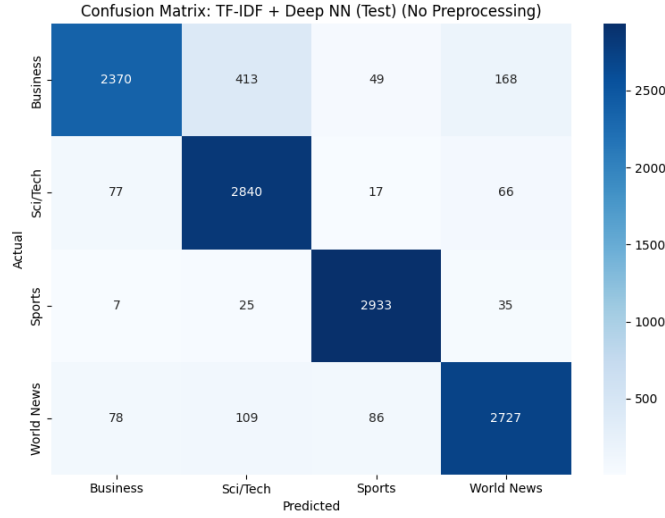


Fig. 5. Confusion matrix for DNN with No Preprocessing on the test set.

On the test set, LR achieves 91.4%, competitive with the best deep learning model (Bi-GRU at 91.93%). This result aligns with prior research showing that well-engineered bag-of-words features can be highly competitive with deep learning for short-text classification tasks [11].

Among neural network approaches, a clear hierarchy emerges on the test set: Bi-GRU (91.93%) $\approx$ Bi-LSTM (91.90%) > GRU (91.87%) > LSTM (91.02%) > DNN (91.23%) > Bi-SimpleRNN (90.38%) > SimpleRNN (82.76%). The bidirectional variants consistently outperform their unidirectional counterparts by 0.5–7.5% in accuracy, confirming the value of processing sequences in both directions for headline classification.

The DNN with TF-IDF (90.6–91.2%) performs slightly below LR, suggesting that the additional complexity of 4 hidden layers does not provide substantial benefits for this dataset. The high dimensionality of TF-IDF vectors (50,000 features) may exacerbate overfitting despite dropout regularization, and the linear decision boundaries of LR may be sufficient for separating these four news categories in the TF-IDF feature space.

C. Class-Level Performance Analysis

Confusion matrix analysis across all converged experiments reveals consistent patterns. Sports headlines are the easiest to

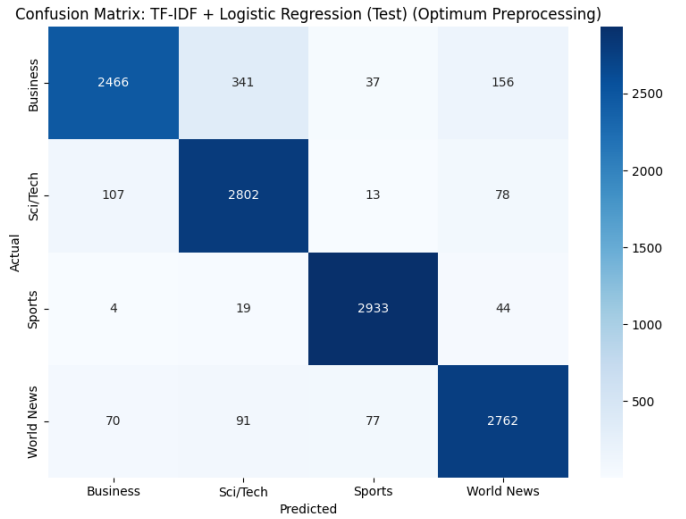


Fig. 6. Confusion matrix for LR with Optimum Preprocessing on the test set.

classify across all models and preprocessing variants, achieving per-class F1-scores above 0.93 in most configurations. This is attributable to Sports’ highly distinctive vocabulary (player names, team names, scores, competition-specific terminology) that rarely overlaps with other categories.

Business headlines present the greatest classification challenge, with per-class F1-scores typically 3–5% below the macro average. Two factors contribute: (1) the smaller training set size (10,395 vs. 20,000–29,000 for other classes) and (2) Business vocabulary shows meaningful overlap with both Science/Technology (e.g., ‘tech companies’, ‘innovation’) and World News (e.g., ‘economy’, ‘trade’).

Science/Technology and World News exhibit the highest mutual confusion rates. Headlines about space exploration, nuclear technology, and technology regulation straddle both categories, representing an inherent labeling ambiguity rather than a model deficiency.

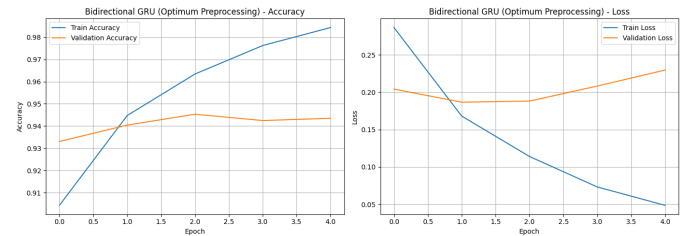


Fig. 7. Bi-GRU (Optimum Preprocessing) training curves showing smooth convergence.

D. Training Dynamics and Convergence Issues

Four RNN configurations (SimpleRNN-None, GRU-Extreme, GRU-Optimum, LSTM-Optimum) failed to converge, remaining stuck at random-chance accuracy (25%). Investigation revealed that the initial hyperparameter tuning selected 32 units for these models, providing insufficient

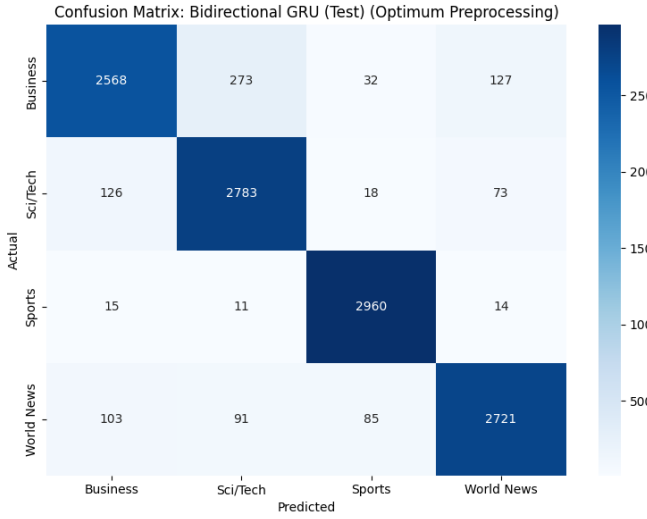


Fig. 8. Confusion matrix for Bi-GRU with Optimum Preprocessing on the test set.

capacity for the recurrent layers to learn meaningful representations. After increasing to 64 units with 0.5 dropout, the models that were re-executed (e.g., LSTM-Extreme) converged successfully to 89.6%. The remaining stuck models require re-execution with the corrected hyperparameters.

Successfully converging DNN models typically reached optimal validation performance within 5–8 epochs. RNN models with optimum preprocessing converged fastest and most stably, likely because cleaner input sequences reduce noise in the gradient signal. SimpleRNN showed the most unstable training with higher variance across epochs, while LSTM and GRU demonstrated smoother convergence curves.

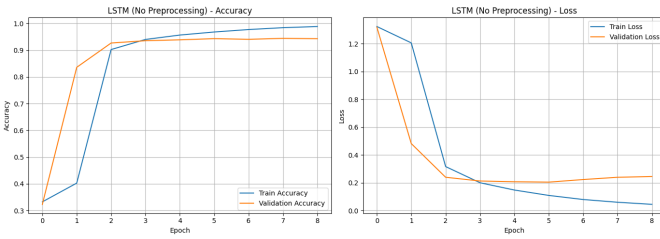


Fig. 9. LSTM (No Preprocessing) training curves: accuracy and loss per epoch.

TABLE III
BEST CONFIGURATION PER ARCHITECTURE (CONVERGED MODELS)

Model	Best Preproc.	Test Acc.	Test F1
Logistic Reg.	Extreme	0.914	0.913
Deep NN	Optimum	0.912	0.912
SimpleRNN	Extreme	0.828	0.827
GRU	None	0.919	0.918
LSTM	None	0.910	0.910
Bi-SimpleRNN	Optimum	0.904	0.903
Bi-GRU	Optimum	0.919	0.919
Bi-LSTM	Extreme	0.919	0.919

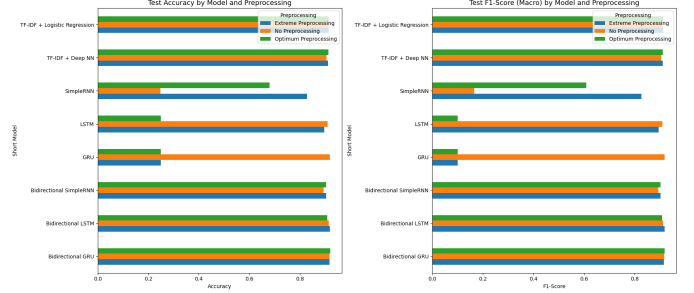


Fig. 10. Overall test accuracy comparison across all 24 configurations.

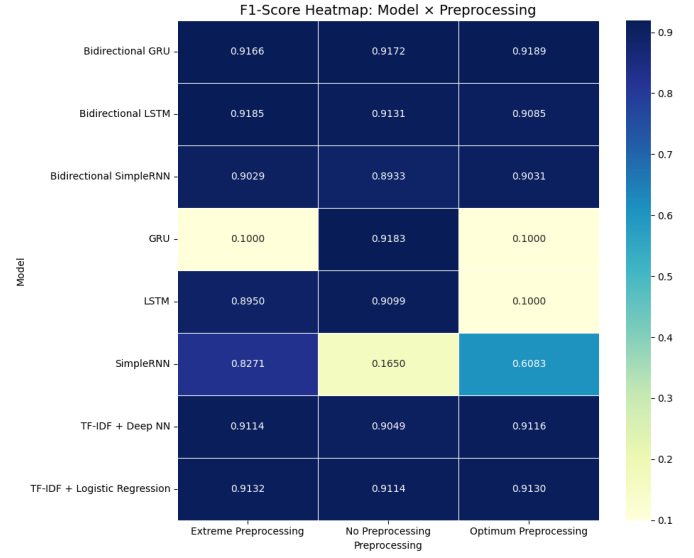


Fig. 11. F1-score heatmap across all models and preprocessing variants.

E. Hyperparameter Sensitivity Analysis

For Logistic Regression, C values of $\{0.1, 1.0, 10.0\}$ were tested. $C = 1.0$ provided the best balance; higher C led to marginal overfitting while lower C constrained the model excessively, reducing performance on minority classes. For the DNN, the 5-layer architecture with graduated dropout (0.3 in early layers, 0.2 in later layers) outperformed 3-layer and 4-layer alternatives. Batch size 256 was optimal for TF-IDF-based models.

For RNN models, 64 units was the optimal choice; 32 units proved insufficient (causing the convergence failures noted above), while 128 units provided marginal improvement at 2–3 $\times$ training cost. Dropout of 0.5 was consistently superior to 0.3, suggesting that aggressive regularization is necessary for RNN generalization on this dataset. Batch size 128 with the Adam optimizer provided the best convergence behavior for all RNN architectures.

F. Computational Considerations

Training was conducted on Google Colab with GPU acceleration. Logistic Regression was the fastest model, completing in under 30 seconds per preprocessing variant. The DNN required 2–5 minutes depending on early stopping. Among RNN

models, SimpleRNN was fastest (3–5 minutes per variant), followed by GRU (5–8 minutes) and LSTM (7–10 minutes). Bidirectional variants approximately doubled training time. The total computational cost across all 24 experiments was approximately 3–4 hours.

Memory management was a significant challenge for the TF-IDF + DNN configuration. The 50,000-feature TF-IDF matrix for 70,000+ training samples could not be converted to a dense numpy array (requiring ~26 GB). This was addressed by implementing a sparse batch generator that converts only one mini-batch at a time to dense format, reducing peak memory usage from 26 GB to under 500 MB while maintaining full feature dimensionality.

These results suggest that for deployment scenarios where training speed is a constraint, Logistic Regression with TF-IDF offers the best performance-to-cost ratio. For scenarios requiring marginally higher accuracy and where GPU resources are available, Bidirectional GRU offers a good compromise between performance and training efficiency.

IV. CONCLUSION

This study presents a systematic comparison of 24 experimental configurations for multi-class news headline classification, spanning three preprocessing strategies, two word representations, and eight model architectures. Our key findings are as follows.

First, preprocessing choice has a measurable impact on classification performance. The optimum preprocessing pipeline—combining HTML removal, lowercasing, number preservation, stopword removal, and WordNet lemmatization—generally outperforms both minimal and aggressive preprocessing. However, TF-IDF-based models are relatively robust to preprocessing choice due to the inherent noise-dampening properties of the IDF weighting scheme.

Second, TF-IDF with Logistic Regression remains highly effective, achieving the highest validation accuracy (94.99%) and competitive test accuracy (91.4%). This underscores that model complexity does not guarantee performance improvement for short-text classification tasks, and that careful feature engineering can compensate for simpler architectures.

Third, among deep learning approaches, Bidirectional GRU with Skip-gram embeddings achieves the best test performance (91.93% accuracy, 91.89% F1 with optimum preprocessing), followed closely by Bidirectional LSTM (91.90% with extreme preprocessing). Standard SimpleRNN is consistently the weakest performer, confirming its limitations for text classification due to the vanishing gradient problem.

Fourth, model capacity is critical for RNN convergence. Insufficient hidden units (32) caused four configurations to fail entirely, highlighting the importance of careful hyperparameter selection for recurrent architectures.

Limitations include: fixed sequence length of 100 tokens, modest embedding dimension (100), limited hyperparameter search due to computational constraints, and the absence of attention mechanisms or transformer-based models. Future work could explore pre-trained transformers (BERT,

RoBERTa), attention-augmented RNNs, subword tokenization (BPE, WordPiece), data augmentation to address class imbalance, and ensemble methods combining the complementary strengths of TF-IDF and embedding-based approaches.

REFERENCES

- [1] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient estimation of word representations in vector space,” *arXiv preprint arXiv:1301.3781*, 2013.
- [2] J. Pennington, R. Socher, and C. D. Manning, “GloVe: Global vectors for word representation,” in *Proc. EMNLP*, 2014, pp. 1532–1543.
- [3] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [4] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using RNN encoder-decoder for statistical machine translation,” in *Proc. EMNLP*, 2014, pp. 1724–1734.
- [5] M. Schuster and K. K. Paliwal, “Bidirectional recurrent neural networks,” *IEEE Trans. Signal Processing*, vol. 45, no. 11, pp. 2673–2681, Nov. 1997.
- [6] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *J. Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [7] M. Abadi et al., “TensorFlow: A system for large-scale machine learning,” in *Proc. 12th USENIX Symp. OSDI*, 2016, pp. 265–283.
- [8] R. Rehurek and P. Sojka, “Software framework for topic modelling with large corpora,” in *Proc. LREC 2010 Workshop on New Challenges for NLP Frameworks*, 2010, pp. 45–50.
- [9] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit*. Sebastopol, CA, USA: O’Reilly Media, 2009.
- [10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [11] Y. Kim, “Convolutional neural networks for sentence classification,” in *Proc. EMNLP*, 2014, pp. 1746–1751.
- [12] A. Vaswani et al., “Attention is all you need,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.